

# ORBIT SHIELD

Gestão de Tráfego Orbital Autônomo

Relatório final de entregas não-código

BIANCA TOLLER - RM553134  
BRUNO MARCELINO - RM553314

FIAP | Global Solution | Indústria Espacial

# Resumo Executivo

Orbit Shield é um MVP acadêmico de gestão autônoma de tráfego orbital. A solução conecta backend, banco Oracle, aplicativo Android nativo e simulação IoT no Wokwi para provar a ideia de um satélite que reduz atraso de reação ao calcular risco localmente.

O backend atua como Mission Control: persiste dados, autentica usuários, expõe API RESTful, busca TLE público, propaga órbita com SGP4 e injeta cenários de detritos. O ESP32 representa o computador embarcado do satélite, calcula aproximação localmente e executa manobra visual. O app Android é o painel do engenheiro.

| Área      | Entrega não-código                              | Arquivo/Evidência                                  |
|-----------|-------------------------------------------------|----------------------------------------------------|
| Banco     | Diagrama ER, modelo relacional, SQL e consultas | docs/database-er-diagram.md; database/oracle/*.sql |
| API       | Swagger, endpoints e arquitetura em camadas     | docs/api-endpoints.md; README.md                   |
| Testes    | Plano, cenários executados e logs               | docs/backend-test-plan.md; docs/iot-test-plan.md   |
| Mobile    | Descrição de telas, arquitetura e integração    | docs/android-setup.md                              |
| Segurança | Login, BCrypt, JWT, perfis e práticas           | README.md; docs/api-endpoints.md                   |
| IoT       | Lógica dos sensores, circuito e autonomia       | docs/orbital-autonomy.md; docs/iot-test-plan.md    |

# Arquitetura Distribuída

A arquitetura foi separada em três partes integradas: Mission Control, Satélite IoT e Painel do Engenheiro. Essa separação demonstra SOA/REST, persistência, segurança, mobile e dispositivos simulados em uma prova de conceito colaborativa.

| Componente | Responsabilidade                                                                     | Tecnologia                                   |
|------------|--------------------------------------------------------------------------------------|----------------------------------------------|
| Backend    | API REST, autenticação, regras de missão, persistência, geração de cenários orbitais | .NET 8, C#, EF Core, Oracle, Swagger         |
| Banco      | Integridade relacional, histórico de sensores, usuários, TLEs e manobras             | Oracle, tabelas OS_*                         |
| IoT        | Computador embarcado simulado, sensores, LCD, LED e servo                            | ESP32 C++ no Wokwi                           |
| Mobile     | Painel nativo do engenheiro com telemetria, alertas e logs                           | Kotlin, Jetpack Compose, Retrofit, StateFlow |

# 1. Banco de Dados

## Modelo e integridade

O modelo relacional usa prefixo OS\_ e contém mais que as 4 entidades mínimas exigidas. As tabelas possuem chaves primárias, estrangeiras, constraints de domínio, unicidade e validações numéricas.

| Tabela                | Finalidade                                  |
|-----------------------|---------------------------------------------|
| OS_USERS              | Usuários, hash de senha e perfil de acesso. |
| OS_SATELLITES         | Satélites monitorados e telemetria base.    |
| OS_DEBRIS_OBJECTS     | Objetos de detrito orbital.                 |
| OS_ORBITAL_ELEMENTS   | TLEs e elementos orbitais.                  |
| OS_CONJUNCTION_EVENTS | Eventos de aproximação e risco.             |
| OS_MANEUVER_LOGS      | Manobras executadas pelo ESP32.             |
| OS_SENSOR_READINGS    | Leituras de sensores simulados.             |

- Diagrama ER: docs/database-er-diagram.md

- DDL Oracle: database/oracle/01\_create\_tables\_current\_user.sql
- Consultas SQL: database/oracle/02\_sample\_queries.sql

## 2. API Backend

A API foi implementada em C#/.NET 8 com organização em camadas. Os controllers chamam serviços de aplicação, que acessam repositórios e persistem dados em Oracle.

| Camada     | Pasta                          | Função                                              |
|------------|--------------------------------|-----------------------------------------------------|
| Controller | src/OrbitShield.Api            | Entrada HTTP, Swagger e autenticação.               |
| Service    | src/OrbitShield.Application    | Regras de negócio, autenticação, cenários orbitais. |
| Repository | src/OrbitShield.Infrastructure | Persistência Oracle com EF Core.                    |
| Domain     | src/OrbitShield.Domain         | Entidades de domínio.                               |

### Endpoints representativos

- POST /api/auth/register e POST /api/auth/login
- GET/POST/PUT/DELETE /api/satellites
- GET /api/satellite/telemetry
- POST /api/satellite/sensor-readings
- POST /api/satellite/maneuver
- GET /api/orbital-scenarios/satellites/1/environment
- POST /api/orbital-scenarios/satellites/1/throw-random-debris

## 3. Testes e Logs

O projeto inclui plano de testes com 7 cenários, superando o mínimo de 5. As evidências foram documentadas com respostas JSON, status HTTP e logs de integração.

| Evidência     | Onde encontrar                           | O que comprova                                                   |
|---------------|------------------------------------------|------------------------------------------------------------------|
| Plano backend | docs/backend-test-plan.md                | Telemetria, emergência, sensores, manobras, autenticação e JWT.  |
| Evidência IoT | docs/iot-test-plan.md                    | ESP32 conecta, envia sensores, calcula risco e registra manobra. |
| Logs Docker   | docker logs orbitshield-api --tail 80    | Requisições, persistência e execução da API.                     |
| Logs túnel    | docker logs orbitshield-tunnel --tail 50 | URL pública usada por Wokwi/celular.                             |
| Logs Wokwi    | Serial Monitor                           | Wi-Fi, POST sensor, GET environment e POST maneuver.             |
| Build Android | scripts/build-android.ps1                | BUILD SUCCESSFUL e APK gerado.                                   |

**Logs são saídas de execução. Eles servem como evidência porque mostram o sistema funcionando: status 201 para sensores, status 200 para ambiente orbital, decisão autônoma do ESP32 e registro de manobra.**

## 4. Mobile Android

O app nativo Android foi criado com Kotlin e Jetpack Compose. Ele segue MVVM/Clean Architecture e consome a API real via Retrofit, sem dados mockados na UI.

| Tela  | Objetivo                       |
|-------|--------------------------------|
| Login | Autenticar engenheiro com JWT. |

| Tela                      | Objetivo                                              |
|---------------------------|-------------------------------------------------------|
| Dashboard                 | Mostrar telemetria, risco orbital, sensores e status. |
| Maneuver Logs             | Exibir manobras gravadas no backend.                  |
| Global / Scenario Control | Disparar debris aleatório e presets reais.            |
| Settings                  | Mostrar sessão, API ativa e saúde do sistema.         |

- Projeto: android/OrbitShieldEngineer
- Guia: docs/android-setup.md
- Build validado com Gradle Wrapper.
- URL do emulador: http://10.0.2.2:5184

## 5. Segurança

A solução inclui autenticação, autorização e práticas de segurança compatíveis com um MVP acadêmico robusto.

- Senha com hash BCrypt, nunca salva em texto puro.
- JWT Bearer para autenticação.
- Perfis Admin, Engineer e SatelliteDevice.
- Endpoints administrativos protegidos por role.
- DTOs e validações de entrada.
- EF Core com consultas parametrizadas para reduzir SQL Injection.
- Segredos locais excluídos do Git por .gitignore e exemplos seguros em .env.example.

## 6. IoT e Autonomia Orbital

O ESP32 no Wokwi representa o computador embarcado de um satélite. O backend fornece ambiente orbital; o satélite simulado decide localmente se precisa manobrar.

| Componente    | Papel na simulação                                  |
|---------------|-----------------------------------------------------|
| ESP32         | Computador embarcado que executa a lógica autônoma. |
| DHT22         | Telemetria térmica enviada ao backend.              |
| Potenciômetro | Thrust/propulsão simulada para cálculo de manobra.  |
| LCD I2C       | Estado visual da missão para a banca.               |
| LED vermelho  | Alerta de risco.                                    |
| Servo motor   | Atuador visual de manobra evasiva.                  |

### Cálculo embarcado

```
tca = -dot(r, v) / |v|^2
closest = r + v * tca
missDistance = |closest|
risk = missDistance <= safeDistance
```

O debris aleatório varia diâmetro, massa estimada, seção transversal, densidade, distância de passagem, velocidade relativa e energia de impacto. A decisão visual fica estável durante o evento para não parecer que múltiplos objetos foram lançados, e volta para SAFE PASS quando o objeto passa.

## 7. Execução Para Demonstração

### Comando principal:

```
powershell -ExecutionPolicy Bypass -File .\scripts\start-demo.ps1
```

- Sobe Oracle, backend e túnel público no Docker.
- Aguarda a API ficar pronta.
- Prepara usuário demo e reseta a missão para SAFE PASS.
- Mostra URL do Swagger, Android Emulator e Wokwi.

### Credenciais Android:

```
API URL: http://10.0.2.2:5184  
email: orbit.demo.2026@orbitshield.local  
password: OrbitShield123!
```

**Fluxo sugerido: iniciar Docker, abrir Swagger, abrir Android, confirmar SAFE PASS, abrir Wokwi, disparar Throw Random Debris, observar dashboard, LCD, LED, servo e logs de manobra.**

## Conclusão

O Orbit Shield atende aos requisitos do desafio: arquitetura distribuída, banco relacional, API RESTful, autenticação, controle de acesso, app mobile nativo, simulação IoT e evidências de teste. A prova de conceito demonstra autonomia orbital em um cenário acadêmico realista, usando ferramentas gratuitas e executáveis localmente.